LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

Disusun Oleh :

Nama	: Diki Hurrisyail
NIM	: (124140088)
Kelas Praktikum	: RA
Pertemuan	: 6
Mata Kuliah	: Pemrograman Berorientasi Objek

Dosen Pengampu:

Arkham Zahri Rakhman, S.Kom., M.Eng.

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNOLOGI INDUSTRI

INSTITUT TEKNOLOGI SUMATERA

2026

I. Tujuan Praktikum

Setelah menyelesaikan praktikum pada bab ini, Mahasiswa diharapkan mampu:

1. Menjelaskan konsep abstract class.
2. Menjelaskan konsep interface.
3. Membedakan abstract class dan interface.
4. Mengimplementasikan abstract class dalam Java.
5. Mengimplementasikan interface dalam Java.
6. Membuat program sederhana yang menggunakan abstract class dan interface.

II. Alat dan Bahan

Pada praktikum ini alat dan bahan yang digunakan terdiri dari:

1. Laptop Windows 11: Sebagai perangkat utama untuk menjalankan, menulis, dan menguji program selama praktikum berlangsung.
2. JDK (Java Development Kit): Sebagai tools utama untuk mengembangkan, meng-compile, dan menjalankan program berbasis bahasa Java.
3. Visual Studio Code: Sebagai text editor atau IDE untuk menulis, mengedit, dan mengelola kode program Java dengan lebih mudah.
4. Modul Praktikum PBO BAB 4: Sebagai panduan dalam memahami materi serta langkah-langkah yang harus dilakukan selama praktikum.

III. Langkah Percobaan

Pada praktikum ini percobaan yang dilakukan seperti yang tertera pada subbab **6.15 Tugas Percobaan** di dalam modul. Berikut langkah percobaan yang dilakukan:

Percobaan 1:

Pada percobaan 1, dibuat abstract class `Hewan` yang memiliki atribut nama, method abstract `suara()`, dan method `makan()`. Kemudian dibuat class `Kucing` dan `Anjing` sebagai turunan yang meng-override method `suara()` sesuai karakteristik masing-masing. Percobaan ini menunjukkan konsep dasar abstract class dan pewarisan.

Kode Program:

```
abstract class Hewan {
    String nama;

    public Hewan(String nama){
        this.nama = nama;
    }

    abstract void suara();
    void makan(){
        System.out.println(nama + " sedang makan.");
    }
}
```

```
class Kucing extends  Hewan{
    public Kucing(String nama){
        super(nama);
    }

    @Override
    void suara(){
        System.out.println(nama + " bersuara: meong meong
meong.");
    }
}
```

```
class Anjing extends Hewan {
    public Anjing(String nama){
        super(nama);
    }

    @Override
    void suara(){
          System.out.println(nama + " bersuara: guk guk
guk.");
    }
}
```

```
public class Main1 {
    public static void main(String[] args) {
        Hewan k = new Kucing("Duta");
        Hewan a = new Anjing("Yowen");

        k.suara();
        k.makan();

        a.suara();
        a.makan();
    }
}
```

Percobaan 2:

Pada percobaan 2, dibuat interface `Terbang` dengan method `terbang()`. Lalu class `Burung` dan `Pesawat` mengimplementasikan interface tersebut dengan cara terbang yang berbeda. Percobaan ini menunjukkan bagaimana interface digunakan untuk merepresentasikan kemampuan yang bisa dimiliki oleh beberapa class.

Kode Program:

```
interface Terbang{
    void terbang();
}
```

```
class Burung implements Terbang{

    @Override
    public void terbang(){
        System.out.println("Burung terbang dengan sayap");
    }
}
```

```
class Pesawat implements Terbang{

    @Override
    public void terbang(){
          System.out.println("Pesawat terbang menggunakan
mesin");
    }
}
```

```
public class Main2 {
    public static void main(String[] args) {
        Terbang b = new Burung();
        Terbang p = new Pesawat();

        b.terbang();
        p.terbang();
    }
}
```

Percobaan 3:

Pada percobaan 3, class `Hewan` digabung dengan interface `Terbang`. Class `Burung` mewarisi `Hewan` sekaligus mengimplementasikan `Terbang`, sedangkan `Kucing` hanya mewarisi `Hewan`. Digunakan instanceof untuk mengecek apakah objek bisa terbang. Percobaan ini menunjukkan kombinasi inheritance dan interface dalam satu program.

Kode Program:

```
abstract class Hewan{
    String nama;

    public Hewan(String nama){
        this.nama = nama;
    }

    abstract void suara();
}
```

```
interface Terbang{
    void terbang();
}
```

```
class Burung extends Hewan implements Terbang {
    public Burung(String nama){
        super(nama);
    }

    @Override
    void suara() {
         System.out.println(nama + " bersuara: cuit cuit
cuit");
    }

    @Override
    public void terbang(){
            System.out.println(nama  +  "  terbang  dengan
sayap.");
    }
}
```

```
class Kucing extends Hewan {
    public Kucing(String nama){
        super(nama);
    }
    @Override
    void suara(){
        System.out.println(nama + " bersuara: meong meong
meong");
    }
}
```

```
public class Main3 {
    public static void main(String[] args) {
        Hewan h1 = new Burung("Atha");
        Hewan h2 = new Kucing("Duta");

        h1.suara();
        if (h1 instanceof Terbang){
            Terbang t = (Terbang) h1;
            t.terbang();
        }
        System.out.println("-------------------------");

        h2.suara();
        if (h2 instanceof Terbang){
            Terbang t = (Terbang) h2;
            t.terbang();
        } else {
               System.out.println(h2.nama  +  "  tidak  bisa
Terbang");
        }
        System.out.println("-------------------------");
    }
}
```

Percobaan 4:

Pada percobaan 4, dibuat sistem sederhana menggunakan abstract class `Hewan` dan interface `Terbang` dengan beberapa subclass seperti `Kucing`, `Anjing`, dan `Burung`. Setiap subclass mengimplementasikan method `suara()`, dan `Burung` juga mengimplementasikan `terbang()`. Objek-objek disimpan dalam array, lalu dilakukan iterasi untuk menampilkan data, suara, serta kemampuan terbang jika dimiliki. Percobaan ini menunjukkan penerapan konsep OOP secara lebih lengkap, termasuk polymorphism, penggunaan array objek, dan pemanfaatan interface dalam kasus nyata.

Kode Program:

```
abstract class Hewan {
    String nama;

    public Hewan(String nama){
        this.nama = nama;
    }

    abstract void suara();

    void tampilkanInfo(){
        System.out.println("Nama: " + nama);
    }
}
```

```
interface Terbang {
    void terbang();
}
```

```
class Kucing extends Hewan{
    public Kucing(String nama){
        super(nama);
    }

    @Override
    void suara(){
       System.out.println(nama + " bersuara meong meong");
    }
}
```

```
class Anjing extends Hewan {
    public Anjing(String nama){
        super(nama);
    }

    @Override
    void suara(){
        System.out.println(nama + " bersuara guk guk");
    }
}
```

```
class Burung extends Hewan implements Terbang{
    public Burung(String nama){
        super(nama);
    }

    @Override
    void suara(){
        System.out.println(nama + " bersuara cuit cuit");
    }

    @Override
    public void terbang(){
        System.out.println("Bisa terbang");
    }
}
```

```
public class Main4 {
    public static void main(String[] args) {
        Hewan[] daftarHewan = {
            new Kucing("Duta"),
            new Anjing("Yowen"),
            new Burung("Atha"),
            new Kucing("Kepin"),
            new Anjing("Agit")
        };

        for (Hewan h : daftarHewan){
            h.tampilkanInfo();
            h.suara();

            if (h instanceof Terbang) {
                Terbang t = (Terbang) h;
                t.terbang();
            } else {
                 System.out.println(h.nama + " tidak bisa
terbang");
            }
            System.out.println("----------------------");
        }
    }
}
```

IV. Hasil Output Program

Percobaan 1:

```
$ java -cp bin Main1
Duta bersuara: meong meong meong.
Duta sedang makan.
Yowen bersuara: guk guk guk.
Yowen sedang makan.
```

Percobaan 2:

```
$ java -cp bin Main2
Burung terbang dengan sayap
Pesawat terbang menggunakan mesin
```

Percobaan 3:

```
$ java -cp bin Main3
Atha bersuara cuit cuit
Bisa terbang
---------------------------------
Duta bersuara meong meong
Duta tidak bisa Terbang
---------------------------------
```

Percobaan 4:

```
$ java -cp bin Main4
Duta bersuara meong meong
Duta tidak bisa terbang
---------------------------------
Nama: Yowen
Yowen bersuara guk guk
Yowen tidak bisa terbang
---------------------------------
Nama: Atha
Atha bersuara cuit cuit
Bisa terbang
---------------------------------
Nama: Kepin
Kepin bersuara meong meong
Kepin tidak bisa terbang
---------------------------------
Nama: Agit
Agit bersuara guk guk
Agit tidak bisa terbang
---------------------------------
```

V. Analisis Program

Pada percobaan 1 sampai 4, class `Hewan` digunakan sebagai abstract class yang berfungsi sebagai kerangka dasar. Di dalamnya terdapat method abstract `suara()` yang wajib diimplementasikan oleh subclass, serta method biasa seperti `makan()` atau `tampilkanInfo()`. Class yang menjadi subclass antara lain `Kucing`, `Anjing`, dan `Burung`, yang masing-masing meng-override method `suara()` dengan perilaku berbeda sesuai jenis hewan. Hal ini menunjukkan konsep polymorphism, di mana satu method memiliki implementasi yang berbeda pada tiap subclass.

Selain itu, digunakan interface `Terbang` yang berisi method `terbang()` untuk merepresentasikan kemampuan terbang. Class yang mengimplementasikan interface ini adalah `Burung` (dan pada percobaan 2 juga `Pesawat`), sedangkan class seperti `Kucing` dan `Anjing` tidak mengimplementasikannya karena tidak memiliki kemampuan tersebut. Penggunaan extends digunakan saat class mewarisi sifat dari abstract class (inheritance), sedangkan implements digunakan saat class ingin memiliki kemampuan tambahan dari interface tanpa harus terikat pada hirarki class tertentu.

Perbedaan perilaku setiap subclass terlihat dari implementasi method `suara()` dan kemampuan tambahan seperti `terbang()`. Misalnya, `Kucing` dan `Anjing` hanya memiliki suara, sedangkan `Burung` memiliki suara sekaligus kemampuan terbang. Abstract class digunakan agar terdapat struktur umum yang harus dimiliki semua hewan, sedangkan interface digunakan untuk menambahkan kemampuan khusus yang tidak dimiliki semua objek. Dengan demikian, program menjadi lebih fleksibel, terstruktur, dan mudah dikembangkan.

VI. Kesimpulan

Berdasarkan percobaan yang telah dilakukan, dapat disimpulkan bahwa penggunaan abstract class dan interface sangat membantu dalam membangun struktur program yang terorganisir. Abstract class digunakan sebagai dasar untuk mendefinisikan atribut dan method umum yang harus dimiliki oleh setiap subclass, seperti pada class `Hewan` dengan method `suara()`. Sementara itu, interface digunakan untuk merepresentasikan kemampuan tertentu yang tidak harus dimiliki semua class, seperti kemampuan `terbang()` pada interface `Terbang`. Dengan konsep ini, setiap class memiliki peran yang jelas sesuai dengan fungsinya.

Selain itu, penerapan inheritance dan polymorphism membuat program menjadi lebih fleksibel dan mudah dikembangkan. Setiap subclass dapat memiliki perilaku yang berbeda meskipun berasal dari class yang sama, seperti perbedaan suara pada `Kucing`, `Anjing`, dan `Burung`. Penggunaan interface juga memungkinkan penambahan fitur tanpa mengganggu struktur utama class. Dengan demikian, kombinasi abstract class dan interface dapat meningkatkan efisiensi, keterbacaan, serta skalabilitas program dalam pengembangan berbasis OOP.